

Sécurité réseau — Security Group & Network ACL

Module Terraform `security_group` et `network` — Projet infrastructure AWS

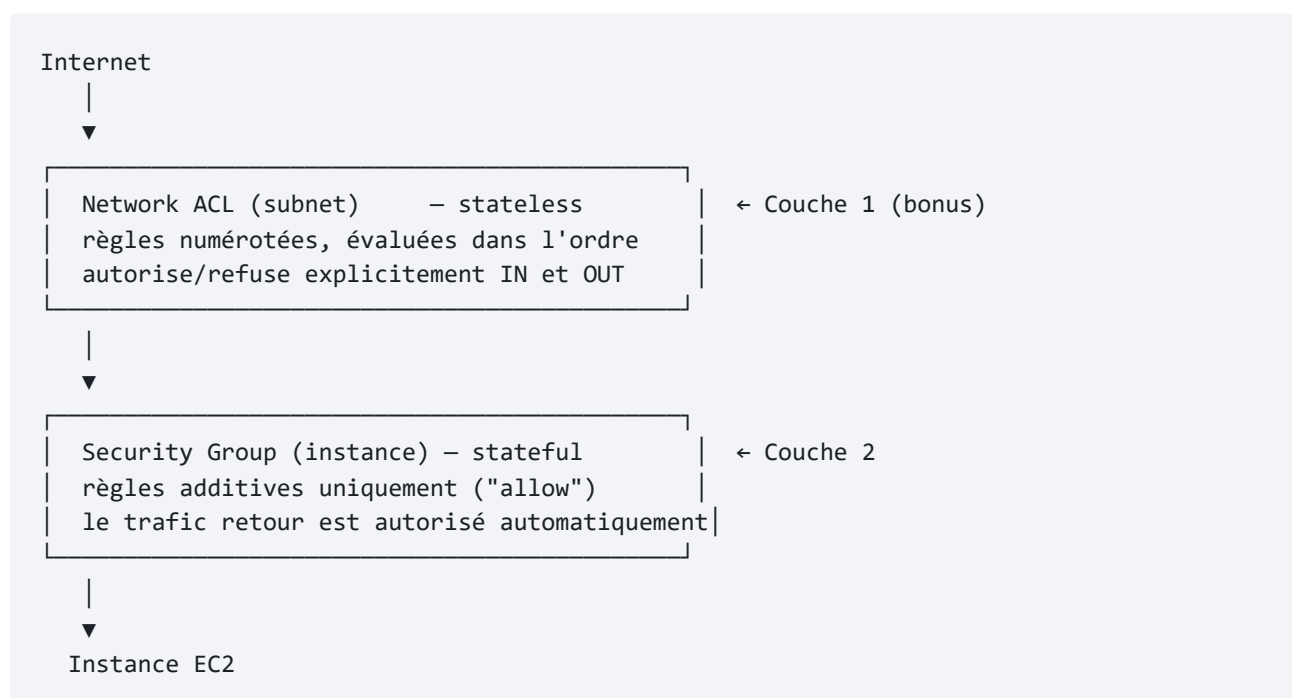
1. Objectif pédagogique

Fail-Safe Default — en cas de doute ou d'absence de configuration explicite, l'accès doit être refusé par défaut, pas autorisé. Une ressource ne doit jamais s'ouvrir "par accident" faute de valeur fournie.

Defense in Depth — la sécurité ne repose jamais sur un seul mécanisme. Plusieurs couches indépendantes doivent filtrer le trafic, pour qu'une erreur de configuration sur l'une n'expose pas directement les ressources.

Ces deux principes sont appliqués concrètement à deux niveaux du réseau AWS : le **Security Group** (pare-feu au niveau de l'interface réseau de l'instance) et la **Network ACL** (pare-feu au niveau du subnet).

2. Architecture — les deux couches de filtrage



Si un attaquant contourne une couche (mauvaise règle de Security Group, par exemple), la Network ACL reste une seconde barrière indépendante. C'est la *defense in depth*.

3. Security Group — `infra/infra/modules/security_group`

Le Security Group est **stateful** : une réponse à une connexion autorisée en entrée est automatiquement autorisée en sortie (et inversement), sans règle explicite à écrire pour le retour.

Sens	Port	Protocole	Source / Destination	Justification
Entrant	80 (HTTP)	TCP	<code>0.0.0.0/0</code>	Trafic web public, ouvert par nature
Entrant	22 (SSH)	TCP	CIDR spécifique (variable <code>ssh_ingress_cidr</code>)	Administration : accès restreint à une IP connue
Sortant	Tous	-1 (tous)	<code>0.0.0.0/0</code>	Écrit explicitement (voir note ci-dessous)

Fail-safe default appliqué à SSH

La variable `ssh_ingress_cidr` n'a **volontairement pas de valeur par défaut** dans le module :

```
variable "ssh_ingress_cidr" {
  type      = list(string)
  description = "CIDR blocks allowed to reach SSH (22). Fail-safe default:
                  no default value, must be set explicitly (e.g. your admin
                  IP/32) rather than falling back to 0.0.0.0/0."
}
```

Résultat : si la personne qui déploie oublie de fournir cette variable, `terraform plan` échoue immédiatement au lieu de créer silencieusement une règle SSH ouverte à tout Internet. L'échec est le comportement sûr par défaut.

Note sur la règle sortante

Quand un Security Group est créé via la console/API AWS, une règle "autoriser tout en sortie" est ajoutée automatiquement. Mais avec Terraform, la ressource `aws_security_group` gère l'intégralité de l'état déclaré : si aucun bloc `egress` n'est écrit, Terraform **supprime** cette règle par défaut plutôt que de la laisser en place. Elle est donc réécrite explicitement dans le code pour obtenir le comportement attendu.

```
resource "aws_security_group" "this" {
  name          = "${local.prefix}-sg"
  description   = "Security group for ${local.prefix}"
  vpc_id        = var.vpc_id

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = var.http_ingress_cidr
  }

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = var.ssh_ingress_cidr
  }

  egress {
    description = "Allow all outbound"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

4. Bonus — Network ACL au niveau subnet

La NACL est **stateless** : contrairement au Security Group, une réponse à une connexion entrante n'est *pas* autorisée automatiquement en sortie. Chaque sens doit être traité indépendamment, avec des règles numérotées évaluées dans l'ordre croissant jusqu'à la première correspondance.

Sens	N° règle	Port	Action	Source
Entrant	100+	80 (HTTP)	allow	http_ingress_cidr
Entrant	200+	22 (SSH)	allow	ssh_ingress_cidr
Entrant	300	1024-65535 (éphémères)	allow	0.0.0.0/0
Sortant	100	Tous	allow	0.0.0.0/0

Pourquoi la règle des ports éphémères ?

Un client qui initie une connexion HTTP ou SSH ouvre localement un port source aléatoire (souvent entre 1024 et 65535). La NACL étant stateless, le *retour* de la réponse vers ce port doit être explicitement autorisé en entrée — sinon la connexion reste bloquée à mi-chemin. C'est le point pédagogique clé qui distingue une NACL d'un Security Group.

```
resource "aws_network_acl" "this" {
  vpc_id      = var.vpc_id
  subnet_ids = [aws_subnet.this.id]
}

resource "aws_network_acl_rule" "ephemeral_ingress" {
  network_acl_id = aws_network_acl.this.id
  rule_number    = 300
  egress         = false
  protocol       = "tcp"
  rule_action     = "allow"
  from_port      = 1024
  to_port        = 65535
  cidr_block     = "0.0.0.0/0"
}
```

5. Résumé

Principe	Où il s'applique
Fail-safe default	<code>ssh_ingress_cidr</code> sans valeur par défaut — force un choix explicite
Defense in depth	Deux pare-feu indépendants : Security Group (instance) + Network ACL (subnet)

Généré pour le module Terraform `infra/infra/modules/security_group` et `infra/infra/modules/network`.